

High Frequency Trades Simulator

Anália Beatriz Ferreira da Cruz¹ RGA 202119060751 , João Pedro de Souza Quintino de Paula² RGA 202019060598

¹Facom – Universidade Federal de Mato Grosso do Sul (UFMS)
Caixa Postal 549 – 79070-900 – Campo Grande – MS – Brazil

analia_beatriz@ufms.br, joao.quintino@ufms.br

Abstract. *This paper describes the development of an Order Book simulator for a cryptocurrency exchange, implemented purely in PostgreSQL 15+. The system addresses classic high-concurrency challenges using row-level locks and PL/pgSQL triggers to ensure the atomicity of financial transactions and prevent deadlocks. For validation, a multiprocessing Python script was developed to generate a 750 MB data payload, proving the system's consistency in handling complex scenarios such as Partial Fills and cryptographic dust containment.*

Resumo. *Este artigo descreve o desenvolvimento de um simulador de Order Book para uma exchange de criptomoedas, implementado puramente em PostgreSQL 15+. O sistema resolve desafios clássicos de alta concorrência utilizando bloqueios a nível de linha e gatilhos em PL/pgSQL para garantir a atomicidade das transações financeiras e evitar deadlocks. Para a validação, desenvolveu-se um script Python multiprocessado responsável por gerar uma carga de dados de 750 MB, atestando a consistência do sistema no tratamento de cenários complexos, como preenchimentos parciais (Partial Fills) e contenção de poeira criptográfica.*

1. Contexto e Justificativa

Em exchanges de criptomoedas, o núcleo operacional do sistema é o Order Book, responsável por organizar as intenções de compra (*Bid*) e venda (*Ask*) de ativos. O principal desafio computacional deste ambiente é o *Matching Engine*, que deve executar transações de forma atômica no exato momento em que os preços se encontram. Este projeto visa solucionar o problema de consistência e concorrência ao centralizar essa regra de negócio inteiramente no banco de dados, lidando com milhares de ordens simultâneas sem gerar condições de corrida, inconsistências de saldo ou *deadlocks* em um ambiente transacional de alta frequência.

2. Modelo Entidade-Relacionamento (ER)

O modelo foi estruturado visando a separação estrita entre capital livre e capital empenhado. A tabela carteiras foi desenhada com controle de **saldo_disponivel** e **saldo_bloqueado**, atuando como entidade associativa entre **usuarios** e **ativos**. A tabela de intenções, **ordens**, contém o estado transicional da operação (aberta, parcial, executada ou cancelada). A liquidação é registrada na tabela imutável **trades**, e o rastreamento temporal dos preços foi modelado na tabela **candles_ohlcv**, utilizando o particionamento nativo do **PostgreSQL BY RANGE** sobre a coluna de minutos para suportar o intenso volume de dados exigido.

3. Regras de Negócio Implementadas

As seguintes regras de negócio foram implementadas no nível do banco de dados:

- 1.Garantia de Saldo: Bloqueio imediato do saldo (reserva) no momento da inserção da ordem, impossibilitando gasto duplo.
- 2.Partial Fill: Suporte a ordens atendidas de forma fracionada por múltiplas contrapartes, mantendo a ordem original ativa até seu preenchimento completo.
- 3.Imutabilidade e Auditoria: Proibição de exclusão ou alteração manual na tabela de trades e o registro automático de ciclo de vida das ordens na tabela auditoria_ordens.

4. Principais Consultas e Gatilhos (Triggers)

A resolução central do problema de concorrência foi implementada através do gatilho **executar_matching()**. Este gatilho PL/pgSQL interage com as ordens opostas utilizando a instrução estrutural:

SELECT * FROM ordens WHERE ... FOR UPDATE SKIP LOCKED

Esta abordagem trava as ordens selecionadas para execução exclusiva daquela thread, sem bloquear a leitura do restante do book, evadindo colisões e deadlocks no PostgreSQL. Além disso, funções aritméticas explícitas (como **ROUND** e **GREATEST**) foram implementadas neste gatilho para corrigir problemas de arredondamento fracionário ("poeira criptográfica") gerados pelos múltiplos partial fills. Consultas em tempo real também foram envelopadas em Views, incluindo o **view_market_summary** e funções de Window Functions para o ranking de movimentação financeira dos usuários nas últimas 24 horas.

5. Saídas da Aplicação

Figure 1. Estatística do Banco Funcional

Dashboard × Properties × Statistics ×

Statistics	Value
Size	746.89 MB
Backends	1
Xact committed	4016
Xact rolled back	5
Blocks read	506445
Blocks hit	350148947
Tuples returned	95239603
Tuples fetched	41212406
Tuples inserted	6479875
Tuples updated	10872694
Tuples deleted	0

Table 1. View view_traders_ranking

Query		Query History		
1	SELECT	*	FROM	public.view_traders_ranking
2	LIMIT	20		
3				
4				

Data Output		Messages		Notifications	
	posicao bigint	usuario_id bigint	nome character varying (100)	volume_24h numeric	
1	1	2536	Trader 2536	2795827.9001970763751803	
2	2	726	Trader 726	2711299.1868213028875456	
3	3	1269	Trader 1269	2683116.8447726183221302	
4	4	1462	Trader 1462	2657336.6040582340516905	
5	5	1225	Trader 1225	2650424.4123095961842289	
6	6	2101	Trader 2101	2640448.9658646292252314	
7	7	745	Trader 745	2625442.8189805166452677	
8	8	1364	Trader 1364	2617683.0628621342872454	
9	9	2163	Trader 2163	2615155.5474465289440078	
10	10	1775	Trader 1775	2611184.6023089742402352	
11	11	1564	Trader 1564	2607558.7384207733774108	
12	12	2361	Trader 2361	2606399.6717193625892745	
13	13	1192	Trader 1192	2606058.1202373828450754	
14	14	524	Trader 524	2602932.5870581586565496	
15	15	694	Trader 694	2600706.6429208985070728	
16	16	46	Trader 46	2598386.6914513831469483	
17	17	940	Trader 940	2595272.8906616953390113	
18	18	872	Trader 872	2591055.6124940047558320	
19	19	2356	Trader 2356	2588860.1000183131637875	
20	20	988	Trader 988	2588447.3871725638042514	

Table 2. View view_market_summary

Query

Query History

1

2

3

4

SELECT * FROM public.view_market_summary

Data Output

Messages

Notifications

Table 3. View view_trades_history

Query

Query History

1

2

3

SELECT * FROM public.view_trades_history

LIMIT 100

Data Output

Messages

Notifications

	id bigint	ativo_base_id character varying (10)	preco numeric (24,8)	quantidade numeric (24,8)	valor_total numeric	criado_em timestamp without time zone
1	1348016	SOL	467.86160365	3.94356754	1845.0438333664855210	2026-06-19 01:10:50.015381
2	1348015	PETR4	20.45983638	271.08038687	5546.2603611873003306	2026-06-19 01:10:50.015381
3	1348014	PETR4	20.45983638	173.91228754	3558.2169475399127052	2026-06-19 01:10:50.015381
4	1348013	PETR4	20.46016828	52.17179093	1067.4436218967777004	2026-06-19 01:10:50.015381
5	1348012	PETR4	20.46059924	63.31193639	1295.4001575841623436	2026-06-19 01:10:50.015381
6	1348011	PETR4	20.46099995	96.80146864	1980.6548450029665680	2026-06-19 01:10:50.015381
7	1348010	PETR4	19.15371967	63.05763531	1207.7882697808335477	2026-06-19 01:10:50.015381
8	1348009	PETR4	19.15347425	5.53707786	106.0542782117551050	2026-06-19 01:10:50.015381
9	1348008	SOL	467.66361823	0.59678801	279.0960400728814223	2026-06-19 01:10:50.015381
10	1348007	SOL	467.65009606	13.15105277	6150.0910911806290862	2026-06-19 01:10:50.015381
11	1348006	SOL	467.62754671	5.55104422	2595.8211902773255162	2026-06-19 01:10:50.015381
12	1348005	SOL	467.62754671	1.22450153	572.6106464165414663	2026-06-19 01:10:50.015381
13	1348004	PETR4	20.46099995	138.32063769	2830.1785608590581155	2026-06-19 01:10:50.015381
14	1348003	PETR4	19.15347425	54.18054234	1037.7456225602247450	2026-06-19 01:10:50.015381
15	1348002	SOL	468.31813930	7.75626183	3632.3981081492129190	2026-06-19 01:10:50.015381
16	1348001	SOL	467.62754671	8.60989618	4026.2246280812005678	2026-06-19 01:10:50.015381
17	1348000	SOL	467.47734988	1.14579440	535.6329296193446720	2026-06-19 01:10:50.015381
18	1347999	SOL	467.41124154	1.51555424	708.3870889396111296	2026-06-19 01:10:50.015381
19	1347998	SOL	467.41124154	9.56847999	4472.4151117765467846	2026-06-19 01:10:50.015381
20	1347997	SOL	467.34779408	2.66865433	1247.1897142875403664	2026-06-19 01:10:50.015381

6. Conclusões e Melhorias Sugeridas

O sistema resolveu com êxito o problema proposto. O uso do bloqueio SKIP LOCKED aliado ao script Python paralelizado provou que o PostgreSQL é capaz de atuar como um Matching Engine resiliente, absorvendo um pico intenso de tuplas geradas em lote e alcançando com estabilidade o volume exigido de aproximadamente 750 MB sem apresentar contenções severas.

Como melhorias sugeridas para o banco de dados, a infraestrutura poderia isolar as inserções de log **auditoria_ordens** em rotinas assíncronas para diminuir o tempo de commit da transação principal. Além disso, a arquitetura atual permite uma fácil conexão direta com ferramentas de Business Intelligence. Como evolução, a base de dados poderia ser conectada ao Power BI utilizando o Power Query para o processo de

ETL, transformando a tabela de Candles e o histórico financeiro em dashboards interativos de análise de mercado.

Referencias

Boulic, R. and Renault, O. (1991) “3D Hierarchies for Animation”, In: New Trends in Animation and Visualization, Edited by Nadia Magnenat-Thalmann and Daniel Thalmann, John Wiley & Sons Ltd., England.

Dyer, S., Martin, J. and Zulauf, J. (1995) “Motion Capture White Paper”, http://reality.sgi.com/employees/jam_sb/mocap/MoCapWP_v2.0.html, December.

Holton, M. and Alexander, S. (1995) “Soft Cellular Modeling: A Technique for the Simulation of Non-rigid Materials”, Computer Graphics: Developments in Virtual Environments, R. A. Earnshaw and J. A. Vince, England, Academic Press Ltd., p. 449-460.

Knuth, D. E. (1984), The TeXbook, Addison Wesley, 15th edition.

Smith, A. and Jones, B. (1999). On the complexity of computing. In *Advances in Computer Science*, pages 555–566. Publishing Press.